COMPONENT DESIGN · PHASE 2

Four coordination drivers, one pure shape.

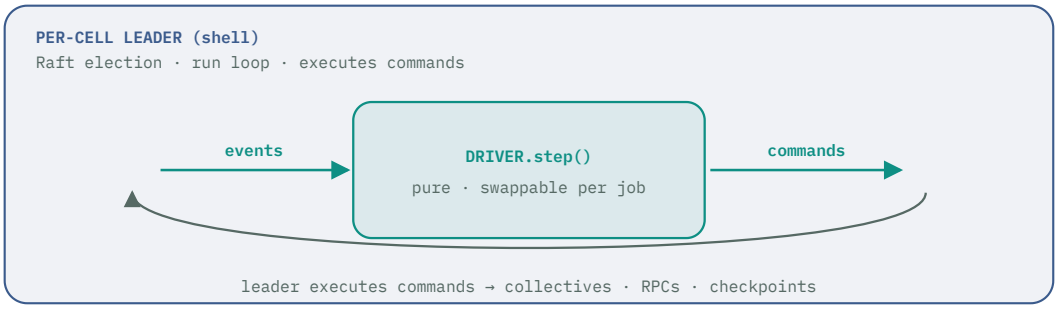
P2 makes the cells actually coordinate. Every driver — barrier, leader, message-passing (independent came free in P0) — is the *same* thing under FCIS: a pure `step(state, event) → (state, commands)` core, hosted by a leader shell that executes the commands. Swap the driver; the loop never changes.

01 One shape, four drivers

The four coordination flavors from the design brief become four instances of one interface. Each defines its own `State`, `Event`, and `Command` sum types, but every one is a pure `step` that folds an event into new state and a list of commands. The **per-cell leader** is the shell that hosts whichever driver a job needs: it gathers events, calls `step`, and executes what comes back.

core/driver.go — the one interface every driver satisfies

```
type Driver interface {
  step(state State, ev Event, now Instant) (State, []Command)  // pure, no I/O
}
// the leader's run loop is driver-agnostic:
//   ev := shell.next()                          // I/O
//   state, cmds = driver.step(state, ev, clock())   // pure
//   shell.exec(cmds)                            // I/O
```



Swap the driver, keep the loop. The leader shell is written once and is driver-agnostic; the coordination logic lives entirely in the pure `step`. That is what lets all four flavors share failure handling, checkpointing, and the run loop.

DRIVER	STATE	EVENT (EXAMPLES)	COMMAND (EXAMPLES)
`independent`	queue cursor	Pulled · Done	Pull · Execute · Report
`barrier`	step N · partials	Done{w,partial} · Deadline	AllReduce · Release · Checkpoint · Evict
`leader`	superstep · assignments	Report{f,result} · RoundTimeout	Assign · Fold · Advance · Reassign
`message-passing`	per-actor state	Message · Crash	Send · Restart

02 The per-cell leader hosts the driver

Per-cell leader CELL · NEW

Responsibility — elect one leader per cell, run the active driver's `step`, and execute its commands. Election is a library concern; the coordination is the pure driver.

◆ FUNCTIONAL CORE · PURE

```
// the hosted driver's step (see §03)
func step(state State, ev Event, now Instant) (State, []Command)
// what to hand a newly-elected leader after a loss
func resume(log []Command, ckpt Checkpoint) State
```

▟ IMPERATIVE SHELL · I/O

- run Raft election (`hashicorp/raft`); replicate the command log
- the run loop: gather events (worker RPCs, timers) → `step` → `exec`
- on leader loss, a new leader calls `resume` from log + last checkpoint

Boundary — events + clock in; new state + commands out. Raft and the network are shell; "what the leader decides" and "how it recovers" are pure, so leader failover is tested by feeding a log + checkpoint to `resume`.

03 The three coordination drivers

Each driver is a pure `step` plus the shell that performs its commands. (`independent` shipped in P0 as the trivial case — no coordination, just the pull queue.)

Barrier driver DRIVER

Responsibility — march every worker in lockstep: compute → wait for all → all-reduce → advance, checkpointing every K and evicting stragglers. This is the P1 spike's core, productionized.

◆ FUNCTIONAL CORE · PURE

```
func step(s Barrier, ev Ev, now Instant) (Barrier, []Cmd)
  // Ev  = Done{w,partial} | Deadline | Restored{ckpt}
  // Cmd = AllReduce{partials} | Release{step}
  //     | Checkpoint{step} | Evict{w} | Rollback{ckpt}
  // all Done → AllReduce + Release; Deadline → Evict;
  // step%K==0 → Checkpoint; member lost → Rollback
```

▟ IMPERATIVE SHELL · I/O

- collect `Done` over gRPC; run the real all-reduce (NCCL / custom)
- set the per-step deadline timer → `Deadline` event
- `Checkpoint` → write shard state to the object store
- `Rollback` → reload the last checkpoint, re-partition

Boundary — worker reports + clock in; collective/checkpoint/evict commands out. The straggler-deadline and checkpoint-cadence rules (B1, B2) are pure — an injected `Deadline` or a missing `Done` is a table test.

Leader driver DRIVER

Responsibility — run coordinated supersteps: assign work, collect reports, fold into global state, advance. Powers `graph-compute`.

◆ FUNCTIONAL CORE · PURE

```
func step(s Super, ev Ev) (Super, []Cmd)
  // Ev  = Report{f,result} | RoundTimeout
  // Cmd = Assign{f,work} | Fold{results}
  //     | Advance{superstep} | Reassign{work}
  // all reports in → Fold + Advance; timeout → Reassign
```

▟ IMPERATIVE SHELL · I/O

- fan `Assign` out to followers over gRPC
- collect `Report`s; drive round timers
- persist folded global state to the replicated log

Boundary — follower reports in; assign/fold/advance out. Follower loss is a pure `Reassign`; leader loss is handled by the host's `resume` (§02).

Message-passing driver DRIVER

Responsibility — deliver messages between actors and supervise them; no global step. Powers `agent-sim`. Here the pure core is *per-actor*, plus a pure router.

◆ FUNCTIONAL CORE · PURE

```
func react(s Actor, m Message) (Actor, []Send)
  // an actor folds one message into new state + outbound
func route(m Message, t RoutingTable) []Delivery
func onCrash(a ActorID) Supervise
  // Supervise = Restart{fromSnapshot} | Escalate
```

▟ IMPERATIVE SHELL · I/O

- deliver `Send`s to mailboxes; drain inbound over the network
- snapshot actors; restart a crashed one from its snapshot
- redeliver in-flight messages (at-least-once)

Boundary — a message + actor state in; new state + sends out. No global ordering, so `react` must be written order-tolerant (rule M1) — property-tested by replaying shuffled/duplicated messages.

04 Checkpointing & mitosis-at-checkpoint

Checkpoint

CELL · NEW

Responsibility — decide when to snapshot, serialize driver state, and restore it. The decision and the (de)serialization are pure; only the store is I/O.

◆ FUNCTIONAL CORE · PURE

```
func due(step int, k int) bool          // cadence
func snapshot(s State) []byte            // serialize
func restore(b []byte) State             // deserialize
```

▟ IMPERATIVE SHELL · I/O

- write `snapshot()` bytes to the object store (S3/MinIO)
- read them back on `Rollback` / `resume`

Boundary — state ⇄ bytes. `restore(snapshot(s)) == s` is the round-trip property test.

Mitosis-at-checkpoint MITOSIS · DELTA FROM P0

Responsibility — enforce brief rule B3: a coupled cell may only split/merge at a checkpoint boundary. P0's `decide` is unchanged; a pure gate defers its commands for coupled cells.

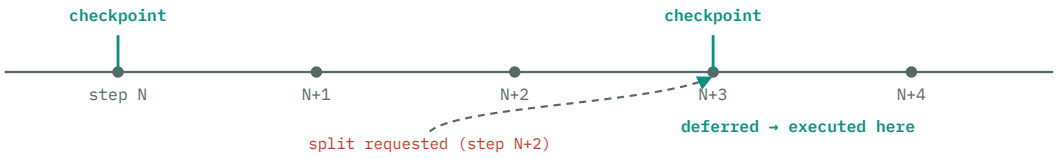
◆ FUNCTIONAL CORE · PURE

```
func resizeSafe(c Coupling, atCkpt bool) bool
  // Independent → always; coupled → only at checkpoint
func gate(m []Mitosis, c Coupling,
          atCkpt bool) []Mitosis
  // pass through, or emit Defer{until: next checkpoint}
```

▟ IMPERATIVE SHELL · I/O

- for a coupled cell, quiesce at the next `Checkpoint`
- then execute the deferred `Split` / `Merge`: snapshot, divide, reload shards, resume

Boundary — P0's `[]Mitosis` + coupling + at-checkpoint flag in; gated commands out. Independent cells (P0/P1) pass straight through; the gate only bites for coupled drivers.



A coupled cell never resizes mid-step. The split is a pure decision the moment it's due, but its *execution* waits for the next checkpoint boundary, where quiescing is safe — so mitosis and lockstep stop fighting.

05 Admission & failure timing

Gang scheduling ADMISSION · DELTA

Responsibility — all-or-nothing admission for coupled jobs (brief B4): place only if `min_members` is satisfiable now, or wait. Extends P0's `admit`.

◆ FUNCTIONAL CORE · PURE

```
func admitGang(job JobSpec, free []CellCapacity) Gang
  // Gang = Place{[]assignment} | Wait
  // Place only if min_members fit now; else Wait
```

▟ IMPERATIVE SHELL · I/O

- on `Place`, atomically reserve all slots
- on `Wait`, hold in a pending queue; retry on capacity change

Boundary — job + free capacity in; `Place`-all or `Wait` out. "128 needed, 100 free → Wait" is a pure test, which is how we prove a gang never starts half-scheduled and deadlocks.

Adaptive-by-tier detection DETECTION · DELTA

Responsibility — set the failure timeout by tier and coupling (brief O4): fast for tight core-tier jobs, patient for the flaky open tier. Pure timing math; the shell only watches the clock.

◆ FUNCTIONAL CORE · PURE

```
func deadline(t Tier, c Coupling) Duration
  // core+barrier → seconds; open+independent → tens of s
func isDead(lastSeen, dl Instant, now Instant) bool
```

▟ IMPERATIVE SHELL · I/O

- track `lastSeen` per member from gossip
- fire an eviction event when `isDead` flips true on a timer

Boundary — tier, coupling, timestamps in; a duration / a boolean out. Both are pure functions of their arguments — no cluster needed to test "a barrier straggler is evicted in seconds."

GPU is a capability, not a component. GPU-bound jobs are handled by extending P0's `place` with a capability filter (match `CapSet` against the task's needs) — a pure predicate. Only the shell touches the device: allocate the GPU and wire up NCCL for the barrier all-reduce.

06 Templates: the driver sequences, the template computes

The four coordinated templates plug into a driver by supplying two pure functions: how to split the job, and how to combine one step's results. The clean split is that the **driver decides *when* to combine** (pure sequencing) and the **template decides *how*** (pure math) — while the shell moves the bytes.

TEMPLATE	DRIVER	DECOMPOSE →	COMBINE (PER STEP) →
`dist-training`	barrier	data shards	all-reduce gradients
`sci-sim`	barrier	spatial domains	boundary exchange
`graph-compute`	leader	vertex partitions	superstep combine
`agent-sim`	message-passing	agent partitions	aggregate state

Three pure layers, one shell. The driver `step` (when), the template `combine` (how), and P0's `decompose` (split) are all pure and independently testable; the leader shell is the only place a gradient actually crosses the wire. A new coordinated template is a new `decompose`/`combine` pair against an existing driver — no shell to write.

07 What P2 defers

P2 completes capability on the **trusted core tier**: all nine templates run, tight jobs included. Still deferred, still over the same boundary: the **open/untrusted tier** — WASM sandboxing plus quorum + reputation verification — in **P3**, and **scale infrastructure** (FoundationDB, observability aggregation, rolling upgrades) in **P4**. Both arrive as new pure cores (a sandbox policy decision; a quorum verdict) plus new shell actions, with the driver `step` interface untouched.